DAA Module: Optimal Control Path Routing via Dijkstra's Algorithm

Technical Report on Hardware Graph Modeling, Min-Heap Priority Queues, & Complexity Proofs

Student Name: Gautham Ram S	**Roll ID / Reg:** S70
Email: gautham.ram.s70@kalvium.community	**Program:** B.Tech Computer Science & Engineering
Live App: https://gauthamram-s70.github.io/Integrated-MicroProject-FLAT-COA-DAA/	**Repository:** https://github.com/gauthamram-s70/Integrated-MicroProject-FLAT-COA-DAA

1. Executive Summary & Curriculum Alignment

This module investigates the application of **Greedy Graph Optimization** to computer hardware design. Aligned with **DAA Modules 1–3 (Algorithm Analysis, Graph Foundations, and Greedy Shortest Paths)**, internal CPU functional execution units are modeled as a Directed Weighted Graph where edge weights quantify physical interconnect latency in nanoseconds. Dijkstra's Algorithm is implemented from scratch with a custom binary Min-Heap to compute minimum-latency routing paths.

Curriculum Connection: *Demonstrates how greedy choice and edge relaxation yield global optimality for non-negative weight networks, achieving O((V + E) log V) time complexity.*

2. Hardware Graph Modeling & Latency Weights

The CPU functional blocks form a directed graph G = (V, E) with |V| = 7 vertices and |E| = 11 interconnect buses:

Source Node (u)	Destination Node (v)	Latency w(u, v)	Physical Bus Function
IF_UNIT	DECODE_UNIT	1.2 ns	Instruction Fetch to Decode Bus
DECODE_UNIT	REG_FILE	0.8 ns	Register File Address Select Lines
DECODE_UNIT	BRANCH_UNIT	1.0 ns	Immediate Offset Bus to Branch Adder
REG_FILE	ALU	1.5 ns	Dual 32-bit Operand Buses A/B
REG_FILE	DATA_MEM	2.2 ns	Direct Store Data Path
ALU	WB_BUS	0.9 ns	Arithmetic Result Forwarding Bypass
ALU	DATA_MEM	1.8 ns	Effective Memory Address Bus
DATA_MEM	WB_BUS	2.0 ns	Data Memory Load Value Bus
WB_BUS	REG_FILE	0.7 ns	Register Write-Back Feedback Bus
BRANCH_UNIT	IF_UNIT	1.1 ns	Branch Redirect PC Update Path

3. Min-Heap Priority Queue & Step-by-Step Relaxation

Edge Relaxation Formulation: For edge (u, v) with weight w(u, v):

```
    if dist[u] + w(u, v) < dist[v] then dist[v] ← dist[u] + w(u, v), prev[v] ← u
```

Using a binary Min-Heap, bubbleUp and sinkDown maintain the heap-order property in O(log V) per operation.

4. Step-by-Step Relaxation Trace for Optimal WB_BUS Path

Step	Action	Vertex / Edge	Relaxation Details & Latency Updates
1	INIT	IF_UNIT	dist[IF_UNIT] = 0.0 ns. All other vertices set to ∞.
2	EXTRACT-MIN	IF_UNIT	Extract u=IF_UNIT (dist=0.0). Relax edge (IF_UNIT → DECODE_UNIT).
3	RELAX	IF → DECODE	dist[DECODE] = 0.0 + 1.2 = 1.20 ns (improved from ∞).
4	EXTRACT-MIN	DECODE_UNIT	Extract u=DECODE (dist=1.2). Relax to REG_FILE and BRANCH_UNIT.
5	RELAX	DECODE → REG	dist[REG_FILE] = 1.2 + 0.8 = 2.00 ns.
6	EXTRACT-MIN	REG_FILE	Extract u=REG_FILE (dist=2.0). Relax to ALU (2.0 + 1.5 = 3.5 ns).
7	RELAX	ALU → WB_BUS	dist[WB_BUS] = 3.5 + 0.9 = **4.40 ns** (Optimal ALU bypass).
8	COMPARE	MEM → WB_BUS	Memory path yields 1.2+0.8+2.2+2.0 = 6.20 ns. Pruned by Dijkstra.

5. Formal Asymptotic Complexity Proof

Theorem: Dijkstra's Algorithm with a Binary Min-Heap terminates in $O((V + E) \log V)$ time.
Proof:
1. Distance array initialization takes $\Theta(V)$ time.
2. Each vertex is inserted once and extracted from the priority queue at most once. Each Extract-Min costs $O(\log V)$. Total extract time: $V \times O(\log V) = O(V \log V)$.
3. Each directed edge (u, v) is relaxed at most once when u is extracted. Each successful relaxation triggers a decrease-key operation in $O(\log V)$. Total relaxation time: $E \times O(\log V) = O(E \log V)$.
4. Summing terms: $T(V, E) = O(V) + O(V \log V) + O(E \log V) = O$**$((V + E) \log V)$**. ■

Data Structure	Extract-Min Time	Decrease-Key Time	Total Time Complexity
Unordered Array	O(V)	O(1)	$O(V^2)$
Binary Min-Heap (Ours)	**O(log V)**	**O(log V)**	**O((V + E) log V)**
Fibonacci Heap	O(log V) amortized	O(1) amortized	O(E + V log V)